

MIE245 - Winter 2026

Lab 6 - Binary Search Trees

Deadline: March 3, 11:59pm

Submission Instructions

Reminder: You can not claim work that was completed by others or generative AI as your own. Submit a completed **readme** template file outlining what code you contributed and what code was completed by other students, people, tools, and/or leveraged from websites.

Add `_utorid` (with `utorid` replaced by your UTORid in all lower-case characters) to the end of both the completed template file names (i.e., the `.py` and the `readme` files). Please ensure you use your UTORid (i.e., what you use to sign into Quercus and **not** other identifiers such as your student number).

Compress your completed template files to a zip file and upload the zip file to Quercus.

1 Lab Instructions

In this lab, you will implement the **insert**, **search**, and **level order traversal** operations for a Binary Search Tree (BST). A level-order traversal of a binary tree is a method of visiting all the nodes present at the same level before visiting the next level. Starting from the root, it visits all nodes at depth 0 first, then all nodes at depth 1, and so on. Your implementation must adhere to the structure provided in the starter code. You may use built-in Python modules or a list to implement a stack or queue.

Sample Code

```
1 from typing import List, Optional
2
3 class TreeNode:
4     def __init__(self, val=0, left=None, right=None):
5         self.val = val
6         self.left = left
7         self.right = right
8
9 class BinarySearchTree:
10     """
11     A class to implement insert, search, and level order traversal for a BST.
12
```

```

13     Methods:
14     -----
15     insert(root: Optional[TreeNode], val: int) -> TreeNode:
16         Inserts a value into the BST and returns the root node.
17
18     search(root: Optional[TreeNode], val: int) -> bool:
19         Searches for a value in the BST and returns True if found, False otherwise
20     .
21
22     level_order_traversal(root: Optional[TreeNode]) -> List[int]:
23         Traverses the BST in level order and returns the node values.
24     """
25     def insert(self, root: Optional[TreeNode], val: int) -> TreeNode:
26         # Implement Insert operation here
27         pass
28
29     def search(self, root: Optional[TreeNode], val: int) -> bool:
30         # Implement Search operation here
31         pass
32
33     def level_order_traversal(self, root: Optional[TreeNode]) -> List[int]:
34         # Implement Level Order Traversal here
35         pass

```

2 Usage Example

Here is a sample Python snippet that shows how the insert, search, and traversal are expected to perform.

```

1 # Inserting values in BST
2 bst = BinarySearchTree()
3 root = None
4 values = [10, 5, 15, 2, 7, 12, 18]
5 for val in values:
6     root = bst.insert(root, val)
7
8 # Insertion creates a tree with the following structure
9 #
10 #           10
11 #          /  \
12 #         5   15
13 #        / \  / \
14 #       2  7 12 18
15
16 # Searching for a value
17 # search_result should be True as 7 is present in the BST
18 search_result = bst.search(root, 7)
19
20
21 # The ordered_list should equal to [10, 5, 15, 2, 7, 12, 18]
22 ordered_list = bst.level_order_traversal(root)

```

AutoGrading

Additionally, you have access to an example of the auto-grading script that will be used to test

your submission (**MIE245_Lab_6_Autograder.py**). Please follow the steps below to run the auto-grader for the provided test case:

1. Place the auto-grader script in a new folder
2. Inside the newly created folder, create a folder called “submissions”
3. Place your script in the “submissions” folder
4. Run the auto-grader script and open the **grades.csv** file to see if you passed the test case (1 is pass, 0 is fail)

Note: The auto-grading script only contains one of the several test cases we will evaluate against, so even if you pass this test case, you may not have a completely correct solution.

Tip: You can modify the test case to test additional functionality.